Voice Cloning and Emotion Detection

Files Covered in This Document

Coqui_TTS_voice_cloning.ipynb
Final_Emotion_Detection_copy.ipynb
Final_Code_Without_SpeechBrain.ipynb
voice_cloning_tts.py

1. System Overview

This document records the full development journey of a voice cloning and emotion detection system built across four code files. The work proceeded in two parallel tracks: a voice cloning pipeline that attempts to reproduce a speaker's voice from a short reference recording, and an emotion detection pipeline that identifies emotional content in audio and uses it to drive reminder generation. Both tracks were developed in Google Colab notebooks with the exception of the final local script.

The development was not linear. Several models were attempted, failed due to dependency conflicts or poor results, and were replaced. This document captures what was tried, what failed, what succeeded, and the current state of each component, based entirely on the code written across the four files.

1.1 The Four Files and Their Roles

File	Role	Environment
Coqui_TTS_voice_cloning.ipynb	Voice cloning with Coqui XTTS models, emotion detection, audio post-processing	Google Colab, GPU
Final_Emotion_Detection_copy.ipynb	Emotion-aware reminder app using SpeechBrain IEMOCAP model with gTTS and Gradio	Google Colab, GPU
Final_Code_Without_SpeechBrain.ipynb	Replacement emotion-aware reminder using Hugging Face Wav2Vec2, no SpeechBrain	Google Colab, GPU optional
voice_cloning_tts.py	Local command-line TTS and voice cloning using torchaudio Tacotron2 and a custom VoiceEncoder	Local machine, no Colab

2. Voice Cloning Development

The voice cloning work is contained entirely in Coqui_TTS_voice_cloning.ipynb and voice_cloning_tts.py. The Colab notebook went through multiple iterations of model selection, audio processing, and emotional control strategy. The local script took a different approach using PyTorch directly without Coqui TTS.

2.1 Prerequisites to Run Voice Cloning

Coqui_TTS_voice_cloning.ipynb

This notebook must run on Google Colab with a GPU runtime. Before running any cell, navigate to Runtime, select Change runtime type, and choose GPU. The notebook installs its own dependencies via pip and apt-get inside the cells.

Package or Tool	How It Is Installed	Purpose in the Code
coqui-tts	pip install coqui-tts	Provides TTS.api.TTS class used to load XTTS v2, YourTTS, VITS, and Tacotron2 models
soundfile	pip install soundfile	sf.read and sf.write used for loading and saving all WAV files
librosa	pip install librosa	Resampling, pitch shifting, time stretching, trim, and spectral feature extraction
transformers	pip install transformers	Loads HuggingFace Wav2Vec2 and HuBERT pipelines for emotion classification
torchaudio	pip install torchaudio>=2.0	Audio loading, Resample transforms, and STFT operations
torch	pip install torch>=2.0	Deep learning backend for all models
matplotlib	pip install matplotlib	Renders emotion confidence bar charts with plt.bar
scipy	pip install scipy	signal.butter and signal.sosfilt used for bandpass EQ and high-pass filters
noisereduce	pip install noisereduce	nr.reduce_noise called in the EmotionalVoiceCloner class enhance_audio method
modelscope	pip install modelscope>=1.14.0	Loads the iic/emotion2vec_plus_large and iic/emotion2vec_base models
funasr	pip install funasr>=1.1.16	Required dependency for the ModelScope Emotion2Vec pipeline
addict	pip install addict	Required indirect dependency flagged during installation debugging
ffmpeg	apt-get install ffmpeg	subprocess.run ffmpeg calls convert MP3, MP4, M4A to WAV and apply audio filters
espeak	apt-get install espeak	Phoneme backend required by some Coqui TTS models during the first stage of development

voice_cloning_tts.py

This script runs on the local machine without Colab. It requires Python 3.8 or higher. Install all dependencies before running with the following command.

```
pip install torch torchaudio numpy sounddevice soundfile librosa
```

Package	Purpose
torch and torchaudio	torchaudio.pipelines.TACOTRON2_WAVERNN_CHAR_LJSPEECH loads Tacotron2 and WaveRNN
numpy	Array math, FFT operations in apply_voice_print, signal generation in fallback_tts
sounddevice	sd.rec captures microphone input, sd.play handles speaker output
soundfile	sf.write saves all audio output as 32-bit float WAV at 22050 Hz

librosa	librosa.effects.pitch_shift and librosa.effects.time_stretch apply emotion effects

2.2 Voice Cloning Model Attempts and Results

The Colab notebook tried multiple TTS and voice cloning models in sequence. Each attempt is preserved in the notebook code. The following table records what was tried and what the code showed about the outcome.

Model Attempted	Model String in Code	Outcome
XTTS v2 (first choice in all later cells)	tts_models/multilingual/multi-dataset/xtts_v2	SUCCESS. Loaded when GPU available. Best quality for voice cloning. Used in all final cells.
YourTTS (second choice fallback)	tts_models/multilingual/multi-dataset/your_tts	SUCCESS as fallback. Lower voice similarity than XTTS v2. Code catches XTTS failure and loads this.
VITS multispeaker (third fallback)	tts_models/en/vctk/vits	SUCCESS as fallback. Basic voice cloning only. Code catches YourTTS failure and loads this.
Tacotron2 DDC (final fallback)	tts_models/en/ljspeech/tacotron2-DDC	SUCCESS for synthesis but NO voice cloning. Generic LJSpeech voice. Used when all others fail.
superb/hubert-large-superb-er (emotion)	HuggingFace pipeline audio-classification	FAILED in some sessions. A fallback using librosa acoustic features was coded and is present.
superb/wav2vec2-base-superb-er (Emotion2Vec)	Wav2Vec2ForSequenceClassification from transformers	PARTIAL. Loaded but output format varied across sessions. Code handles output parsing with multiple branches.
iic/emotion2vec_plus_large (ModelScope)	ModelScope Tasks.emotion_recognition	FAILED due to funasr and addict dependency conflicts in early attempts.
iic/emotion2vec_base (fixed fallback)	ModelScope Tasks.emotion_recognition base model	SUCCESS after switching to base model and pinning modelscope==1.14.0 and funasr==1.1.16.

2.3 How to Run the Voice Cloning Notebook

Coqui_TTS_voice_cloning.ipynb - Step by Step

1. Open the notebook in Google Colab. Set runtime to GPU via Runtime then Change runtime type.
2. Run the installation cells. These install coqui-tts, soundfile, librosa, transformers, torchaudio, scipy, noisereduce, modelscope, funasr, and addict.
3. Run the imports cell. All libraries including TTS, librosa, soundfile, numpy, torch, torchaudio, matplotlib, scipy, and warnings are imported.
4. The model loading section tries XTTS v2 first, falls back to YourTTS, then VITS, then Tacotron2. The first one that loads without raising an exception is used for the session.
5. Run the upload cell. The Colab file dialog opens. Upload a WAV or MP3 reference audio file. The file is saved as reference_input.wav.
6. The preprocessing cell detects the file extension. If MP4, MP3, or M4A, ffmpeg converts it to WAV with mono channel and the specified sample rate. The audio is then loaded with soundfile, collapsed to mono if stereo, and resampled to 16000 Hz using librosa.resample. The cleaned file is saved as reference_fixed.wav or reference_enhanced.wav depending on the cell version.
7. The emotion detection cell loads the classifier and runs it on the reference audio. If the HuggingFace classifier fails, detect_emotion_fallback runs librosa acoustic feature analysis and maps pitch, energy, spectral centroid, and tempo to emotion scores. Results are plotted with matplotlib.
8. The user is prompted to enter the text to synthesise and to choose a desired emotion. The text is modified by get_emotional_text or add_vocal_expressivity based on the chosen emotion.

9. tts.tts_to_file is called with text, speaker_wav set to the reference audio path, file_path set to the output filename, and language set to en. In the XTTS cells, temperature is also passed.
10. Emotion effects are applied to the output using apply_emotion_to_audio or apply_professional_emotion_effects depending on the cell. The processed audio is saved as a new WAV file.
11. The generated audio is played back with IPython.display.Audio and the file is offered for download using files.download.

2.4 Key Settings and Variables in the Voice Cloning Notebook

Variable or Parameter	Location in Code	Value and Meaning
tts_model_name	First model-load cell	tts_models/multilingual/multi-dataset/xtts_v2. The Coqui model string used to initialise TTS.
text_to_speak	First synthesis cell	A string entered by the user or defaulting to a preset sentence. Passed directly to tts_to_file.
language	tts_to_file call	en. Sets the synthesis language for XTTS.
temperature	tts_to_file in XTTS cells	0.6 to 0.95 depending on candidate index. Controls synthesis diversity.
speed	tts_to_file in EmotionallyAccurateCloner	0.8 to 1.2 depending on emotion. Passed to XTTS to control speech rate.
target_sr (reference audio)	Preprocessing cells	16000 Hz for emotion detection input, 22050 or 24000 Hz for TTS reference input.
num_candidates	synthesize_with_emotion method	4 to 6. Number of XTTS outputs generated with varied seeds and temperatures.
TARGET_EMOTION	Final EmotionallyAccurateCloner cell	String set directly in the cell, for example happy. Drives candidate selection.
top_db in librosa.effects.trim	enhance_audio methods	25 to 30. Silence threshold for trimming reference audio before feeding to TTS.
afftdn=nf=-20 in ffmpeg	subprocess.run ffmpeg calls	Noise floor in decibels for ffmpeg audio denoising filter applied during conversion.

2.5 Emotion Post-Processing Applied to Voice Cloning Output

After generating the base cloned voice with tts_to_file, several cells in the notebook apply additional DSP effects to shift the emotional quality of the audio. These are applied with librosa and scipy to the output WAV file.

Emotion	Pitch Shift (n_steps)	Time Stretch Rate	Volume Factor	Additional Effect
happy	+4.0	1.3x faster	1.2x louder	Bandpass EQ boost at 2000 to 8000 Hz
sad	-3.0 to -2.5	0.7 to 0.85x slower	0.85x softer	3 Hz tremolo applied via sine modulation
angry	+1.0	0.95x	1.4 to 1.8x louder	Midband EQ boost at 500 to 3000 Hz, tanh distortion
excited	+6.0 to +4.0	1.5 to 1.35x faster	1.25 to 1.4x louder	No additional DSP
fearful	+2.0	1.15x faster	0.75x softer	6 to 8 Hz tremolo applied via sine modulation
neutral or calm	0.0	1.0x	1.0x	No effects applied
romantic (in later cell)	+1.5	0.85x slower	0.90x softer	tanh soft compression at 0.8 factor

2.6 EmotionallyAccurateCloner: The Final Voice Cloning Strategy

The last major iteration of the Colab notebook introduced the EmotionallyAccurateCloner class. This replaced the single-candidate generation approach with a multi-candidate strategy to avoid relying on post-processing DSP alone. The class generates four to six TTS outputs with different seeds, temperatures, and text variants, scores each one using Emotion2Vec Plus via ModelScope, and selects the candidate with the highest probability for the target emotion. The class also adds speed variation via the speed parameter passed to tts_to_file, which was added in the final version of the synthesize_with_emotion method.

2.7 How to Run the Local Script: voice_cloning_tts__1_.py

12. Open a terminal. Navigate to the folder where voice_cloning_tts.py is saved.
13. Run the script with: python voice_cloning_tts.py
14. The script checks for GPU and prints the device. The VoiceEncoder network is initialised randomly. The Tacotron2 and WaveRNN models are loaded from torchaudio.pipelines.TACOTRON2_WAVERNN_CHAR_LJSPEECH. If that fails, tacotron2 is set to None and the fallback sine-wave synthesiser is used.
15. The output directory Desktop/ChatterBox_Audio_Output is created automatically using os.makedirs.
16. A five-option menu appears. Option 1 generates speech with emotion. Option 2 records three seconds from the microphone, extracts a 256-dimensional voice embedding via VoiceEncoder, synthesises speech, and applies the embedding. Option 3 runs a test generation with no user input. Option 4 opens the output folder. Option 5 exits.

2.8 Local Script Settings and Architecture

Setting or Component	Value or Description
Sample rate	22050 Hz used for all recording, synthesis, and file output
VoiceEncoder architecture	Three Conv1d layers (1 to 64 to 128 to 256 channels, kernel 5, stride 2), bidirectional LSTM (512 hidden), Linear (1024 to 256). Weights are random. Not pretrained.
Voice embedding application	apply_voice_print computes np.fft.rfft on audio, multiplies by 1 plus 0.1 times the mean of the voice embedding, then irfft. Placeholder implementation only.
Fallback TTS	Generates a sine wave at a fixed frequency. Duration scales with text character count at 0.15 seconds per character. Not intelligible speech.
Output filenames	Option 1: chatterbox_emotion.wav. Option 2: cloned_voice_emotion.wav. Option 3: test_audio_YYYYMMDD_HHMMSS.wav
Emotion: happy	pitch_shift n_steps 0.2, time_stretch rate 1.1, fallback 220 Hz
Emotion: sad	pitch_shift n_steps -0.2, time_stretch rate 0.9, fallback 160 Hz
Emotion: angry	pitch_shift n_steps 0.3, time_stretch rate 1.2, fallback 190 Hz
Emotion: excited	pitch_shift n_steps 0.4, time_stretch rate 1.3, fallback 250 Hz
Emotion: calm	pitch_shift n_steps -0.1, time_stretch rate 0.95, fallback 180 Hz
Emotion: neutral	No pitch or speed change, fallback 200 Hz

3. Emotion Detection Development

The emotion detection work is contained in two notebooks: Final_Emotion_Detection_copy.ipynb which uses SpeechBrain, and Final_Code_Without_SpeechBrain.ipynb which replaces SpeechBrain with a Hugging Face Wav2Vec2 model. Both notebooks expose their functionality through a Gradio interface. The SpeechBrain version was the first attempt. When it produced dependency conflicts, the second notebook was written to replace it.

3.1 Final_Emotion_Detection_copy.ipynb: SpeechBrain Version

Overview

This notebook implements an emotion-aware voice reminder application. A user uploads or records a short voice clip. The application detects the emotion in the clip using the SpeechBrain IEMOCAP Wav2Vec2 classifier, maps the raw emotion label to one of four simplified tones, generates a spoken reminder using gTTS, and plays back the audio. An extended version of the same notebook also clones the user's voice using XTTS v2 and delivers the reminder in the user's own voice.

Prerequisites

A GPU runtime is recommended but not required. Navigate to Runtime, Change runtime type, and select GPU.

```
pip install speechbrain gradio gTTS torch torchaudio
pip install coqui-tts   (required only for the voice cloning pipeline)
```

The SpeechBrain model speechbrain/emotion-recognition-wav2vec2-IEMOCAP is downloaded automatically from Hugging Face Hub the first time EmotionDetector is instantiated. It requires internet access and downloads the model weights and a custom_interface.py file.

How to Run

17. Set runtime to GPU in Colab if available.
18. Run Cell 1: pip install speechbrain gradio gTTS torch torchaudio.
19. Run Cell 2: the full application is defined in one cell. It defines EmotionDetector using speechbrain.inference.interfaces.foreign_class, defines VoiceReminder using gTTS, defines the analyze_and_remind Gradio callback, builds the Gradio interface using gr.Blocks, and calls demo.launch with share set to True.
20. A public Gradio link appears. Click it to open the app in a browser.
21. In the browser interface, upload or record a voice clip in Step 1. Optionally type a custom reminder in Step 2. Click the button.
22. The results show the raw SpeechBrain emotion label, the confidence score, the mapped tone, the reminder text that was spoken, and the generated MP3 audio file.

EmotionDetector Class

The EmotionDetector class calls foreign_class with source set to speechbrain/emotion-recognition-wav2vec2-IEMOCAP, pymodule_file set to custom_interface.py, and classname set to CustomEncoderWav2vec2Classifier. The predict method calls self.classifier.classify_file with the audio path. The return value is a four-element tuple: out_prob, score, index, and text_lab. The label is extracted from text_lab, normalised to lowercase, and looked up in label_to_tone.

Emotion Label Mapping

SpeechBrain Raw Label	Mapped Tone	Reminder Speed
angry	sad	Normal
sad	sad	Normal

frustrated	sad	Normal
fear	sad	Normal
disappointed	sad	Normal
happy	happy	Normal
excited	excited	Normal
surprised	excited	Normal
neutral	neutral	Normal

VoiceReminder Class

The build_text method returns the custom_text if provided. Otherwise it looks up DEFAULT_REMINDERS by tone key. The speak_to_file method creates a gTTS object with lang set to en and slow set to True only when tone equals sad. The output is saved to the reminders directory as reminder_tone_randomhex.mp3.

Default Reminder Texts

Tone	Default Reminder Text
sad	Hey, I hear some heaviness in your voice. Take a deep breath and remember to be kind to yourself.
happy	You sound cheerful. Keep that energy going and remember to share your happiness with others.
neutral	Thanks for checking in. Stay focused, stay calm, and keep moving forward one step at a time.
excited	You sound excited. Channel that energy into something amazing today.

3.2 Why SpeechBrain Was Replaced

The SpeechBrain notebook encountered dependency conflicts when pip attempted to install speechbrain alongside coqui-tts in the same Colab session. SpeechBrain requires specific versions of torchaudio and soundfile that conflict with the versions required by Coqui TTS. Additionally, the foreign_class loader requires the custom_interface.py file to be downloaded alongside the model weights. In some Colab sessions this download failed silently, causing classify_file to raise an attribute error at runtime.

As a result, a second notebook was written that removes all SpeechBrain dependencies. This became Final_Code_Without_SpeechBrain.ipynb.

3.3 Final_Code_Without_SpeechBrain.ipynb: Replacement Version

Overview

This notebook implements the same emotion-aware voice reminder application but replaces the SpeechBrain classifier with the Hugging Face pipeline for audio-classification using the model ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition. The rest of the application, including VoiceReminder, gTTS output, and the Gradio interface, is identical in structure. The Gradio interface uses the same field layout and the analyze_and_remind callback follows the same three-step pattern: detect emotion, build reminder text, generate audio.

Prerequisites

GPU runtime is recommended. Install all packages by running the single installation cell.

```
pip install gradio gTTS torch transformers soundfile librosa
```

The model ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition is downloaded automatically from Hugging Face Hub when EmotionDetectorHF is instantiated. It is approximately 1.2 GB and is cached after the first download within the session.

How to Run

23. Open the notebook in Colab. GPU runtime is optional but recommended.
24. Run Cell 1: pip install gradio gTTS torch transformers soundfile librosa.
25. Run Cell 2: the entire application runs in one cell. EmotionDetectorHF is defined, VoiceReminder is defined, analyze_and_remind is defined, the Gradio interface is built with gr.Blocks, and demo.launch is called with share set to True.
26. Click the Gradio public link printed in the output.
27. Upload or record audio. Optionally enter custom text. Click the button to run.

EmotionDetectorHF Class

This class creates a Hugging Face pipeline with task set to audio-classification, model set to ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition, and device set to 0 if CUDA is available or -1 for CPU. The predict method calls self.pipe with the audio file path and top_k set to 1. It extracts the label and score from the top prediction and looks up the label in label_to_tone.

Emotion Label Mapping

Hugging Face Raw Label	Mapped Tone	gTTS slow Parameter
angry	sad	False
disgust	sad	False
fear	sad	False
sad	sad	True
happy	happy	False
surprise	excited	False
calm	neutral	False
neutral	neutral	False

3.4 Gradio Interface Fields - Both Emotion Notebooks

Field Name	Type	Description
Step 1 - Upload or record voice	Audio input	Accepts microphone recording or file upload. Passes the file path to analyze_and_remind.
Step 2 - Custom reminder text	Textbox, optional	If non-empty, this text is passed to speak_to_file instead of the default tone reminder.
Detected raw emotion	Textbox, read only	The raw label string returned by the model, such as angry, calm, or surprise.
Model confidence	Textbox, read only	The probability score formatted to two decimal places.
Mapped tone	Textbox, read only	One of Sad, Happy, Neutral, or Excited, capitalised.
Reminder text that was spoken	Textbox, read only	The exact string passed to gTTS.
Voice reminder audio	Audio output	The generated MP3 file. Can be played in the browser or downloaded.

4. Current Development State and Known Issues

The following section records the current state of every major component across all four files as of the time of writing. It includes what is working, what failed and was replaced, and what remains incomplete at this stage of development.

4.1 Component Status Summary

XTTS v2 voice cloning (Coqui notebook) [SUCCESS] Loads and synthesises with speaker reference when GPU is available. Best results require 5 to 15 seconds of clean reference audio.

YourTTS fallback (Coqui notebook) [SUCCESS] Loads as fallback when XTTS fails. Voice similarity lower than XTTS v2.

VITS fallback (Coqui notebook) [SUCCESS] Basic synthesis only. Used when both XTTS and YourTTS fail.

Tacotron2 fallback (Coqui notebook) [SUCCESS] No voice cloning. Produces generic LJSpeech voice. Final fallback.

superb/hubert-large-superb-er emotion (Coqui notebook) [PARTIAL] Fails to download in some Colab sessions. Fallback using librosa acoustic heuristics is active.

iic/emotion2vec_plus_large (ModelScope) [FAILED] Dependency conflicts between funasr, addict, and existing Colab packages prevented loading.

iic/emotion2vec_base (ModelScope, fixed version) [SUCCESS] Loads after pinning modelscope==1.14.0 and funasr==1.1.16. Used in the final EmotionallyAccurateCloner cell.

SpeechBrain IEMOCAP classifier (Final_Emotion_Detection_copy.ipynb) [PARTIAL] Works in isolation but fails when coqui-tts is also installed due to torchaudio version conflicts.

Hugging Face Wav2Vec2 emotion (Final_Code_Without_SpeechBrain.ipynb) [SUCCESS] Loads reliably. Replaces SpeechBrain in the final emotion detection pipeline.

gTTS reminder generation (both emotion notebooks) [SUCCESS] Requires internet. Generates MP3 with slow parameter for sad tone. No other prosody control.

Gradio interface (both emotion notebooks) [SUCCESS] Launches with share=True. Public link valid for 72 hours before expiring.

Tacotron2 WaveRNN via torchaudio (local script) [PARTIAL] Loads on recent torchaudio versions. Some versions fail. Fallback sine wave synthesiser activates automatically.

VoiceEncoder voice cloning (local script) [INCOMPLETE] Weights are randomly initialised. apply_voice_print applies only a mean-based frequency scaling. Not a real voice clone.

4.2 Dependency Conflicts Encountered

Conflict	Packages Involved	Resolution in Code
SpeechBrain and Coqui TTS torchaudio versions clash	speechbrain, coqui-tts, torchaudio	Written as two separate notebooks so they are never installed together
emotion2vec_plus_large fails to initialise	modelscope>=1.14.0, funasr>=1.1.16, addict	Switched to iic/emotion2vec_base and added addict to the install list
TTS import fails after pip install	TTS==0.22.0 vs TTS==0.20.3 and TTS==0.21.3	Later cells pin to TTS==0.20.3 or TTS==0.21.3 and import with a try-except block
ModelScope output format changes across versions	modelscope, funasr	emotion_scores method has multiple if-isinstance branches to handle dict and list output formats
jedi version incompatible with IPython in Colab	jedi, IPython	Later install cells add jedi>=0.16,<0.19 and specific fsspec and gcsfs versions
ffmpeg not available in base Colab	ffmpeg-python, system ffmpeg	subprocess.run with ffmpeg is used rather than a Python library, and apt-get installs the binary

4.3 Audio Processing Standards Across All Files

Processing Step	Tool Used	Parameters

Reference audio upload (Colab notebooks)	google.colab files.upload	Saves raw bytes to disk. Handles WAV, MP3, MP4, and M4A
Format conversion (Colab notebooks)	subprocess.run ffmpeg	Converts to mono WAV. Sample rate 16000, 22050, or 24000 Hz depending on model. Filters: highpass f=80, lowpass, afftdn nf=-20
Silence removal (Colab notebooks)	librosa.effects.trim	top_db between 25 and 30
Normalisation (Colab notebooks)	librosa.util.normalize	Scales to peak amplitude 1.0
Noise reduction (EmotionalVoiceCloner cells)	noisereduce nr.reduce_noise	prop_decrease 0.7 to 0.8
STFT noise reduction (earlier cells)	librosa.stft and librosa.istft	Spectral subtraction using first 50 frames as noise estimate
Bandpass EQ (emotion post-processing)	scipy.signal.butter and sosfilt	Butterworth filter order 2 to 4. Bands vary by emotion.
Microphone recording (local script)	sounddevice sd.rec	Duration 3 seconds for voice cloning, sample rate 22050 Hz, channels 1, dtype float32
Output format (all files)	soundfile sf.write	PCM 16-bit WAV in notebooks, 32-bit float WAV in local script

4.4 What Remains to Be Done

- The VoiceEncoder in voice_cloning_tts.py has no trained weights. It needs to be pretrained on a speaker verification dataset such as VoxCeleb before the voice embedding carries any meaningful speaker identity.
- The apply_voice_print method applies only a scalar frequency scaling based on the mean of the embedding. A proper implementation would use the full embedding to condition a neural vocoder.
- The fallback emotion detector in the Coqui notebook uses acoustic heuristics that do not generalise well. It should be replaced with a smaller locally-cached model.
- The Gradio public link expires after 72 hours. A persistent deployment using Colab with a tunnelling service or a local server would be needed for production use.
- The gTTS reminder in both emotion notebooks has no true prosody control beyond the slow parameter. Integration of the XTTS model into the reminder generation step would allow the reminder to be delivered in the user's own voice with appropriate emotional tone.
- The emotion-to-tone mapping in both notebooks uses only four output tones: sad, happy, neutral, and excited. Expanding this to the full set of model labels would improve specificity.